

NetworkPeers — Product & Engineering Change Specification

Multi-Worker Jobs · Collectionist/Correctionist Roles · Capture QA Pipeline · Admin Scope · Client Review

NetworkPeers Engineering

Revision 2 — September 2026 (OCR Preview & Capture-Quality Enforcement Update)

What changed in Revision 2: This revision tightens three requirements that were implicit or partial in Revision 1: (1) every image submission — in the worker's own history, the correctionist queue, and the client review screen — must show its OCR output as a first-class preview, not just in the correctionist/client review panes; (2) the on-device edge-to-edge / boundary check is now a hard, non-optional gate — a capture that is not edge-to-edge, or that otherwise fails the quality filter, is rejected at capture time with no path to upload; (3) the client review screen is specified in full as a split-screen layout — submitted image on one side, generated OCR text on the other — with an independent 'view full' / expand control on each half. Section-by-section changes are marked “UPDATED (Rev. 2)” below; net-new subsections are marked “NEW (Rev. 2)”.

Contents

- 0. Purpose of This Document
- 0.1 Current State (as observed on the live build)
- 1. Change 1 — A Job Can Be Accepted by Infinite Workers
- 2. Change 2 — Worker Roles: Collectionist vs. Correctionist
- 3. Change 3 — In-App Preview + Automatic Open-Source Image Quality Filter
- 4. Change 4 — Admin Portal Scope Decision
- 5. Change 5 — Client Review: Per-Page View With OCR, Approve/Reject
- 6. NEW (Rev. 2) — OCR Preview Everywhere & Split-Screen Full-View Mode
- 7. Cross-Cutting Architecture Summary
- 8. Master QA / Definition-of-Done Checklist

0. Purpose of This Document

This document is the single source of truth for a set of interlocking changes to the NetworkPeers platform (web app at `network-peer-web.vercel.app`, API at `network-peer-api-alpha.vercel.app`, Android app, iOS app, and the `rudraaxl/NetworkPeer` monorepo). It is written so that any teammate — web, Android, iOS, or backend — can pick up their slice of the work without needing a live walkthrough.

Each change is documented with: what changes and why (product rationale), data model impact (shared contract / API changes), backend changes, web (TypeScript / Next.js) changes, Android (Kotlin) changes, iOS (Swift) changes, and an edge-cases & QA checklist. The last sections cover cross-cutting architecture, the recommended open-source image-quality pipeline, a suggested rollout order, and open decisions that need sign-off (marked [DECISION NEEDED]).

0.1 Current State (as observed on the live build)

Based on the deployed site and repo structure, the platform today has three surfaces:

Surface	URL	Purpose today
Client portal	/client	Post jobs with media-required checklists, track workers live, approve evidence
Worker app	/worker	Find nearby jobs, capture photo/video/audio proof in-app, get paid
Admin console	/admin	Operations, payouts, disputes, analytics, fraud dashboard

Core existing product principles that must be preserved:

- Anonymous by default — no names/photos/phone/email exposed until a job is accepted.
- In-app capture only — no gallery uploads. This constraint is reused and extended (see §3) rather than removed.
- GPS + timestamp proof on every evidence item.
- Fraud scoring (duplicate hashes, device fingerprints, network signals).
- Escrow payments, released on evidence approval.

Repository layout (`rudraaxl/NetworkPeer`):

```
NetworkPeer/
├── apps/           # web (TS/Next.js), android (Kotlin), ios (Swift)
├── docs/           # product & architecture docs – this file belongs here
├── infra/terraform/ # infra as code
├── packages/contracts/ # shared API/data contracts consumed by all 3 clients
├── Dockerfile
└── package.json    # workspace root (monorepo, npm workspaces)
```

The presence of `packages/contracts` is important: it is the shared contract package (likely OpenAPI/JSON-schema or TS types) that both the web app and, indirectly, the mobile apps' networking layers should be generated from or kept in sync with. Every data-model change below should land in `packages/contracts` first, then be consumed by the three apps, to avoid drift between Kotlin/Swift/TS models.

1. Change 1 — A Job Can Be Accepted by Infinite Workers

1.1 What changes and why

Today a job is implicitly a 1:1 contract — one worker accepts, does the work, submits evidence, gets paid. For data-collection style jobs (e.g., “scan every page of this book,” “photograph every shelf in this store,” “collect 500 samples of X”) a single worker is a bottleneck and a single point of failure.

New behavior: a job can define `maxWorkers`: `null` | `number`. When `maxWorkers` is `null` (or a sentinel like `-1` / “unlimited”), the job stays open for acceptance indefinitely — any number of workers can accept it concurrently and each performs their own unit of work against the same job (e.g., their own subset of pages/units). Jobs with a normal worker cap keep today’s 1:1 (or 1:N capped) behavior unchanged.

This is the foundational change that makes §2 (roles) and §5 (per-page client review) meaningful — a single job now produces many submissions, each independently reviewable.

1.2 Data model impact (packages/contracts)

Add/modify the Job entity and introduce a JobAssignment (or JobAcceptance) entity — this is the key structural change: assignment must become its own row/entity, not a field on Job.

```
// packages/contracts/src/job.ts
export type WorkerCapacityMode = "single" | "capped" | "unlimited";

export interface Job {
  id: string;
  title: string;
  description: string;
  jobType: "capture" | "collection" | "correction" | "mixed"; // see §2
  capacity: {
    mode: WorkerCapacityMode;
    maxWorkers: number | null; // null when mode === "unlimited"
  };
  unitOfWork?: {
    kind: "page" | "item" | "location" | "freeform";
    totalUnits?: number; // e.g., total pages in the book, if known upfront
  };
  status: "draft" | "open" | "in_progress" | "closed" | "archived";
  // ...existing fields: price, location, mediaRequirements, escrowId, etc.
}

export interface JobAssignment {
  id: string;
  jobId: string;
  workerId: string;
  role: "collectionist" | "correctionist"; // see §2
  acceptedAt: string; // ISO timestamp
  status: "active" | "submitted" | "withdrawn" | "removed";
  unitsClaimed?: string[]; // e.g., page numbers/ids this worker is responsible for
}
```

Key contract rule: `Job.capacity.mode === "unlimited"` disables the existing “acceptance lock” logic that today flips job status to assigned / `in_progress` and hides it from other workers’ nearby-jobs feed. Unlimited jobs stay visible in the nearby-jobs feed even while workers are actively working on them, until the client or admin manually closes the job (or `totalUnits` is fully claimed, if known).

1.3 Backend changes

- Acceptance endpoint (POST `/jobs/:id/accept`): remove the “already assigned” 409 rejection when `job.capacity.mode === "unlimited"`. Instead, create a new JobAssignment row per accepting worker; never block on existing assignments.

- Concurrency control: for capped jobs, use an atomic counter/transaction (SELECT ... FOR UPDATE or Redis INCR compare-and-set) to prevent over-acceptance — generalize the existing 1:1 lock to “ $N \leq \text{maxWorkers}$ ”.
- Nearby-jobs feed query: filter out jobs where `mode !== "unlimited"` AND `assignmentCount >= maxWorkers`, instead of filtering out any job with ≥ 1 assignment.
- Job detail response: include `assignmentCount` and (for the client) a roster of active workers (still anonymized to the worker's own view — see anonymity rules).
- Escrow: escrow amount semantics change — for unlimited jobs, either (a) price is per unit of work and escrow tops up dynamically, or (b) a fixed pool is split pro-rata. [DECISION NEEDED] — recommend (a), per-unit pricing.
- Notifications/websocket events: broadcast `job.assignment.created` to the client's live-tracking view (§5) so new workers appear on their dashboard in real time.

1.4 Web (TypeScript) changes

- Job-creation form (/client → new job flow): add a “Worker capacity” control — radio group: Single worker / Limited (N workers) / Unlimited (open collection job).
- Job card / job detail components: replace the binary “Open / Assigned” badge with a live count, e.g. “14 workers active”, when `mode === "unlimited"`.
- Client live-tracking view: change from a single-worker map pin/status timeline to a roster list component (scrollable list of active workers + their submission counts) — see §5/§6 for the review UI that hangs off this.

1.5 Android (Kotlin) / iOS (Swift) changes

- Nearby-jobs list screen: job cards must render worker-capacity state (“Unlimited · 14 joined” vs “3/5 spots filled” vs “Taken”).
- Accept-job flow: remove any client-side disabling of the “Accept” button based on a cached “already assigned” flag for unlimited jobs; always attempt the accept call and handle the (now rarer) 409 only for capped jobs.
- Local models: `Job.kt` / `Job.swift` need the `capacity` and `unitOfWork` fields added, generated/mirrored from packages/contracts.
- Active-jobs screen: a worker can now see multiple simultaneously active unlimited jobs they've joined.

1.6 Edge cases & QA checklist

- Two workers accept the same “single” job within the same millisecond → only one succeeds (race condition test).
- Unlimited job with `totalUnits` set (e.g., 300-page book) auto-transitions to closed when all units are claimed/submitted-and-approved, even with capacity technically still “unlimited.”
- Client can manually close an unlimited job early — remaining active assignments should be allowed to finish and submit, not force-cancelled.
- A worker cannot accept the same job twice (idempotency on (jobId, workerId) pair).
- Payout math is verified for both per-unit and pooled escrow models before going live.

2. Change 2 — Worker Roles: Collectionist vs. Correctionist

2.1 What changes and why

Introduce two explicit worker roles for data-oriented jobs:

- Collectionist — captures raw data (e.g., photographs each page of a physical document/book, or captures other structured field data). This is the existing capture flow, extended with the QA pipeline in §3.
- Correctionist — reviews data already collected by a collectionist (image + the OCR/extraction result run against it) and makes a binary quality-control decision: Redo or Approve.

This creates a two-stage human pipeline: capture → verify, which dramatically improves final data quality without needing the client to review every single page themselves (though the client retains override power — see §5).

2.2 Data model impact

```
// packages/contracts/src/worker.ts
export type WorkerRole = "collectionist" | "correctionist";

export interface WorkerProfile {
  id: string;
  // ... existing fields
  eligibleRoles: WorkerRole[]; // a worker can be approved for one or both
  activeRole?: WorkerRole;     // role selected in-session, if job supports both
}

// packages/contracts/src/submission.ts
export interface Submission {
  id: string;
  jobId: string;
  assignmentId: string; // links back to JobAssignment (collectionist's)
  unitRef: string;      // e.g., "page-014" or item id
  mediaUrl: string;     // the captured image
  ocrResult?: {
    engineVersion: string;
    text: string;
    confidence: number; // 0-1
    boundingBoxes?: OcrBox[];
    generatedAt: string;
  };
  qualityCheck: QualityCheckResult; // see §3.2 – auto filter result at capture time
  status: "pending_review" | "approved" | "redo_requested" | "client_approved" |
"client_rejected";
  reviewHistory: ReviewEvent[];
}

export interface ReviewEvent {
  id: string;
  submissionId: string;
  reviewerRole: "correctionist" | "client" | "admin";
  reviewerId: string;
  decision: "approve" | "redo" | "reject";
  note?: string;
  createdAt: string;
}
```

Design notes: Submission is deliberately decoupled from JobAssignment — a correctionist reviewing the job is not the same assignment as the collectionist who captured it. Model the correctionist's work as its own JobAssignment with role: "correctionist" on the same jobId, and their ReviewEvents reference the collectionist's Submissions.

Job.jobType (§1.2) gates which roles are relevant: a "collection" job only needs collectionists; a "correction" job (or a "mixed" job) opens both roles. Recommend most real jobs are "mixed": the job auto-generates correction work as collection submissions land, keeping one job id but two visible role queues.

2.3 Backend changes

- New endpoint: GET /jobs/:id/review-queue?role=correctionist — returns Submissions with status: "pending_review" for that job, each bundled with its ocrResult.
- OCR pipeline hook: when a collectionist's capture is uploaded and passes the on-device quality filter (§3), the backend triggers OCR asynchronously and attaches ocrResult to the Submission, flipping status to pending_review once OCR completes (recommend an async UI with a “processing...” state so correctionists/clients/the worker aren't blocked — see §6).
- Review endpoint: POST /submissions/:id/review with { decision: "approve" | "redo", note?: string }. approve → status: "approved" (pending the client's own approval per §5). redo → status: "redo_requested"; re-opens the specific unit back into the collectionist queue for any collectionist to reshoot — sane precisely because of §1 (unlimited acceptance).
- Role eligibility: an admin/vetting step (or in-app qualification quiz) should gate who can act as a correctionist, since correctionist decisions gate payouts. [DECISION NEEDED] — minimum bar for correctionist eligibility.

2.4 Web (TypeScript) changes — UPDATED (Rev. 2)

Clarified requirement: The correctionist review screen is the reference implementation for OCR preview: every submission the correctionist opens must render the captured image and its generated OCR text together, and each half must be independently expandable to a full-screen view. This same pane is reused, unmodified in structure, for the client review screen in §5 and the OCR-preview requirement in §6.

- Worker-facing web preview (/worker) and the real worker experience (native apps) both need a role switcher at the top of a job detail screen when jobType === "mixed" : two tabs/buttons — “Collect” / “Correct” — each leading to a different sub-flow.
- Correctionist review screen — build once, share the design across web/Android/iOS:
 - Split-pane (or stacked-on-mobile) layout: image viewer on one side, OCR text output on the other, scroll-synced where practical.
 - Image viewer: pinch/scroll zoom, pan, rotate — correctionists need to inspect small text/detail. A dedicated “View full image” control opens the image alone, edge-to-edge, in a full-screen lightbox/modal.
 - OCR panel: raw extracted text, with low-confidence tokens visually highlighted (underlined/colored where per-token/line confidence is below a threshold). A dedicated “View full text” control opens the OCR panel alone, full-screen, for reading/selecting/copying long passages without the image competing for space.
 - Two big, unambiguous controls: “Approve” and “Redo” (no 3rd “reject” state here — reject is reserved for the client tier in §5).
 - Optional short note field on “Redo”.
 - Queue behavior: after a decision, auto-advance to the next pending submission in the queue.

2.5 Android (Kotlin) changes

- New CorrectionistReviewFragment/Compose screen mirroring the web split-pane: ZoomableImageView (or Compose zoomable modifier / PhotoView) for the image half, and a scrollable Text/LazyColumn for OCR output with confidence-based span styling. Each half gets its own expand-to-fullscreen affordance, matching §2.4.
- Role selection persisted in local session/DataStore so a returning correctionist lands back in “Correct” mode on that job.
- Offline queue consideration: correctionist decisions should queue locally (Room DB) and sync when connectivity returns, same pattern as evidence upload retries.

2.6 iOS (Swift) changes

- New CorrectionistReviewViewController / SwiftUI CorrectionistReviewView: UIScrollView with minimumZoomScale/maximumZoomScale (or MagnificationGesture) for the image, paired with a Text/AttributedString OCR panel highlighting low-confidence spans. Same independent full-screen expand control on each half.

- Same “Approve” / “Redo” two-button control, same auto-advance-to-next-in-queue behavior.
- Persist activeRole in UserDefaults/Core Data alongside existing job-session state.

2.7 Edge cases & QA checklist

- A correctionist cannot review their own collectionist submission (self-review prevention).
- “Redo” correctly re-queues only the specific unit, not the whole job.
- OCR still processing → correctionist queue shows a clear “processing” state rather than an empty/broken card (see §6.2 for the shared processing-state spec).
- Concurrent correctionists don't both grab and review the same submission (claim-on-open with timeout release).
- Low-confidence highlighting threshold is configurable per job/document type.

3. Change 3 — In-App Preview + Automatic Open-Source Image Quality Filter

3.1 What changes and why — UPDATED (Rev. 2)

Today capture is “in-app only, no gallery uploads” — good. This change adds two things before final upload:

- 1. A preview screen after the shot is taken, so the worker can see exactly what was captured before committing it.
- 2. An automatic, on-device, open-source quality filter that runs against the captured frame and checks framing / edge-to-edge coverage, blur/focus quality, and exposure/glare.

Hard gate (Rev. 2 clarification): This is a hard reject, not a warning. If the quality filter flags the image as “dirty” on any check — most importantly, if the document/page is not captured edge-to-edge (a corner or side is cut off, or the boundary contour is not fully closed inside the frame) — the capture is rejected immediately on-device, before it is ever queued for upload. The worker sees the specific reason and must retake; there is no “upload anyway” override available to the worker. This is the same behavior described in §3.8’s flow diagram, restated here explicitly because it is a hard product requirement, not an optional nicety.

3.2 Data model impact

```
// packages/contracts/src/quality.ts
export interface QualityCheckResult {
  passed: boolean;
  checks: {
    edgeCoverage: { passed: boolean; score: number; message?: string };
    sharpness:    { passed: boolean; score: number; message?: string }; // e.g., Laplacian
    variance
    exposure:     { passed: boolean; score: number; message?: string };
  };
  overallScore: number; // 0-1 composite
  engineVersion: string; // for later re-tuning/telemetry
  ranOnDevice: boolean;  // true = client-side pre-check; false = server-side re-check
  checkedAt: string;
}
```

Submission.qualityCheck (§2.2) stores the result of the check that passed at capture time, for audit/traceability. Also store rejected attempts (lightweight telemetry) — how often workers fail the filter, which check fails most — to tune thresholds over time without needing production incidents to notice a threshold is miscalibrated.

3.3 Recommended open-source approach (client-side, on-device)

Run checks on-device, before upload, so the worker gets instant feedback and no bad image ever leaves the phone.

3.3.1 Edge-to-edge / document boundary detection — UPDATED (Rev. 2)

Core technique: classic OpenCV document-scanner pipeline — grayscale → Gaussian blur → Canny edge detection → findContours → pick the largest 4-point contour → check that the detected quadrilateral’s area is close to the full frame area and that no corner touches the image boundary in a way indicating the true edge is cut off (the signature of a cropped/missing-edge shot is a boundary contour that is open/off-frame rather than a closed quadrilateral fully inside the captured image).

Reject condition (explicit): Treat this check as pass/fail, not advisory. Reject the capture if: (a) no 4-point contour is found above the confidence threshold, or (b) the contour’s bounding box covers less than ~90–92% of the frame (page too small / not edge-to-edge), or (c) two or more contour corners sit within a few pixels of the frame’s outer edge (indicating the page extends beyond what was captured — i.e., it was not shot edge to edge). Any of these three conditions is sufficient on its own to reject; the worker is shown which condition failed and is returned directly to the camera.

- Android: OpenCV Android SDK (org.opencv:opencv-android) for Canny/findContours/approxPolyDP, or Google ML Kit Document Scanner API (purpose-built “scan a document edge-to-edge” UX with a built-in capture UI — evaluate first).

- iOS: VNDocumentCameraViewController (VisionKit) as the primary path — live edge detection + auto-capture + perspective correction, built into iOS and free. For custom-UI needs, fall back to OpenCV or VNDetectRectanglesRequest.
- Web (fallback, e.g. admin re-upload): opencv.js (official OpenCV WASM build) runs the same Canny/contour pipeline in-browser.

3.3.2 Sharpness / blur detection

Technique: variance of the Laplacian — grayscale → Laplacian kernel → variance of the result; low variance ⇒ blurry. Reuses the OpenCV dependency already pulled in for §3.3.1. Threshold tuned empirically per job type.

3.3.3 Exposure / glare check

Technique: histogram analysis on the grayscale image — flag if a large fraction of pixels are clipped at 0 (too dark) or 255 (too bright/glare), or if overall mean brightness is far outside an acceptable band. Uses OpenCV calcHist.

3.3.4 Why this combination

- Open source and free (OpenCV = BSD/Apache-family license, ML Kit Document Scanner and VisionKit = free) — no per-scan API cost, works offline, keeps the check on-device.
- Cheap enough to run in real time on-device, enabling instant “recapture now” feedback rather than a round trip to the server.
- Reusable across both platforms with the same conceptual pipeline, keeping Android/iOS behavior consistent even though the concrete libraries differ.

3.4 Backend changes

- Server-side re-validation: even though checks run on-device, re-run a lightweight version of the same checks server-side on upload (defense in depth against a modified/rooted client, and to catch anything the on-device pass missed). Store the result in Submission.qualityCheck with ranOnDevice: false.
- Upload endpoint: reject (422) uploads whose server-side re-check fails hard thresholds, with a structured error body identical in shape to the client-side QualityCheckResult.
- Telemetry endpoint: POST /telemetry/quality-check to log rejected-at-capture events (pass/fail per check, score) even when the worker never uploads, so thresholds can be tuned from real-world data.

3.5 Web (TypeScript) changes

- If the web app has any capture surface (worker preview embed, or an admin manual-upload path), integrate opencv.js behind a lazy-loaded chunk (large WASM bundle — only load it on capture screens).
- Build a shared QualityCheckResult → user-facing message mapping component (e.g., "Page edges not fully visible — please recapture the whole page", "Image is too blurry — hold steady and retake", "Too much glare — try angling the page away from the light") so the same copy/logic ports to Kotlin/Swift string resources.

3.6 Android (Kotlin) changes

- Add org.opencv:opencv-android (or ML Kit Document Scanner) as a Gradle dependency.
- Capture flow: CameraX capture → Preview screen (retake/use-photo) → on “use photo,” run quality filter synchronously (brief spinner) → FAIL: inline rejection reason + auto-return to camera (no manual back-navigation required) → PASS: proceed to upload with QualityCheckResult attached.
- Run OpenCV checks on a background thread (Dispatchers.Default); target sub-second feedback.
- Unit tests around threshold functions using saved sample images (good/blurry/cropped/glare) as fixtures.

3.7 iOS (Swift) changes

- Prefer VNDocumentCameraViewController where it fits the job type (prevents many “cropped/missing edge” failures before the shutter fires). For custom control, use AVFoundation capture + preview + VNDetectRectanglesRequest/OpenCV.

- Preview screen: same UX as Android — big preview image, Retake / Use Photo buttons.
- Run checks off the main thread with a lightweight progress indicator; surface failures via shared copy with Android/web.
- Same background telemetry call and retry/upload flow as Android, sharing the QualityCheckResult JSON shape from packages/contracts.

3.8 UX flow summary (all platforms) — UPDATED (Rev. 2)

```

Capture shot
|
Preview screen (worker sees full image, can Retake or Continue)
|
Continue -> run on-device quality filter (edge-to-edge, sharpness, exposure)
|
+-- FAIL (incl. "not edge-to-edge") --> show specific reason(s) + "Retake" prompt
|   --> back to Capture. HARD BLOCK: never reaches upload/network call.
|
+-- PASS --> attach QualityCheckResult --> upload to server
|
|   Server re-validates (defense in depth)
|   |
|   OCR pipeline triggered (§2.3) --> Submission enters correctionist queue
|   AND appears with its OCR preview in the worker's own submission
|   history (§6) as soon as OCR completes

```

3.9 Edge cases & QA checklist — UPDATED (Rev. 2)

- Filter thresholds tuned separately per unitOfWork.kind (a book page vs. a storefront photo need different edge-coverage expectations — storefront jobs likely shouldn't run the edge-to-edge check at all).
- Confirm the edge-to-edge check is enforced as a hard reject in every capture entry point (worker app camera flow, any web-based capture/re-upload surface) — there must be no code path that allows an image with an open/cut-off boundary contour to reach the upload endpoint from the client.
- Worker with a low-end/older device: verify quality-check runtime stays acceptable (benchmark on a representative low-spec Android device).
- Retake loop doesn't hard-block a worker forever — after N consecutive failures, offer a “flag as impossible to capture” escape hatch that routes to a human (admin) rather than trapping the worker.
- On-device pass but server-side re-check fail — make sure the UI can handle a late rejection gracefully (post-upload notification, not silent).
- Verify the filter doesn't produce excessive false positives on legitimately good images — budget real QA time with sample images from actual field conditions.

4. Change 4 — Admin Portal Scope Decision

4.1 What changes and why

The admin console (/admin) currently covers operations, payouts, disputes, analytics, and fraud detection. The ask: it does not need to ship as a customer-facing/production-critical surface in every build, but should remain available, restricted to the internal team, for operational access (payouts, dispute resolution, monitoring the collectionist/correctionist pipeline, quality-filter telemetry review from §3.4).

4.2 Recommendation

Treat this as a build-configuration and access-control decision, not a feature-deletion decision:

- Keep the admin console in the codebase and deployed, but gate it behind stronger auth (internal SSO / allow-listed accounts, not just an admin role flag).
- Ensure it is not linked from public marketing/landing surfaces — remove the footer “Admin access” link and use a direct, unlisted URL or internal-only subdomain instead.
- Do not ship any admin functionality inside the native mobile apps at all — admin stays a web-only, internal tool.
- Add the new operational views this change set requires: quality-filter telemetry dashboard (§3.4), collectionist/correctionist pipeline monitor, role-eligibility management.

[DECISION NEEDED]: Final call on public visibility (keep the footer “Admin access” link vs. remove it) rests with the product owner. The technical recommendation above (keep the code, tighten access, drop the public footer link) satisfies both “may not be needed for the build” and “kept only for our access.”

4.3 Backend / infra changes

- Move admin API routes behind a distinct auth guard/middleware (a separate role: "internal_admin" check rather than reusing a general role: "admin" flag that might overlap with job-poster/“client-admin” concepts).
- Consider deploying admin as a separate Vercel project/subdomain so it can be firewalled/IP-allow-listed independently of the public client/worker surfaces.

4.4 Web (TypeScript) changes

- Remove (or move behind an internal-only nav) the Admin access link currently on the public landing page.
- Add the three new dashboard views listed in §4.2 under the existing /admin route tree.

4.5 Android / iOS changes

- None — confirm no admin-only screens exist in the native codebases; if any do, remove/deprecate them as part of this change.

5. Change 5 — Client Review: Per-Page View With OCR, Approve/Reject

5.1 What changes and why — UPDATED (Rev. 2)

The client currently approves “evidence” at a job level. With multi-worker unlimited jobs producing many page-level submissions (§1, §2), the client needs the same kind of granular review UI the correctionist has (§2.4's split image+OCR view), but at the client tier, with the client able to override a correctionist's approval — i.e., the client is the final authority, functioning like a “super-correctionist” for their own job.

Restated as an explicit requirement: On the website, the client review screen must present, for every submitted page: the submitted image and the OCR text generated from that image, side by side in a split-screen layout. Each side must have its own “view full” control — clicking it expands just that pane (image or OCR text) to full screen/full width, independent of the other pane. This is specified in full in §6 below, which is the single reference for how this UI behaves everywhere it appears (correctionist queue, client review, worker's own submission history).

5.2 Data model impact

Reuses Submission and ReviewEvent from §2.2 — no new entities required, just a new reviewer role value already modeled (reviewerRole: "client"). Add client-facing aggregation fields on Job for convenience:

```
// extension to Job (§1.2)
export interface JobReviewSummary {
  totalUnits: number;
  collected: number;
  correctionistApproved: number;
  clientApproved: number;
  clientRejected: number;
  redoRequested: number;
}
```

5.3 Backend changes

- GET /jobs/:id/submissions?status=&page=&pageSize= — paginated, filterable submission list for the client's per-page review UI, each item bundled with its image URL, OCR result, and correctionist review history.
- POST /submissions/:id/review (same endpoint as §2.3) — accepts reviewerRole: "client" with decision "approve" | "reject" (a genuine reject, distinct from correctionist's redo, since a client rejection may carry payment/dispute consequences).
- GET /jobs/:id/review-summary → returns JobReviewSummary for a dashboard header (e.g., “212 / 300 pages client-approved”).
- Payment/escrow hook: [DECISION NEEDED] — recommend correctionist approval releases payout to the collectionist (keeps workers paid promptly), while a subsequent client rejection triggers the existing dispute/refund flow rather than blocking payout up front.

5.4 Web (TypeScript) changes — UPDATED (Rev. 2)

New client-facing route, e.g. /client/jobs/[id]/review, reusing the same split-pane image+OCR component built for correctionists in §2.4:

```
<SubmissionReviewPane
  submission={...}
  mode="correctionist" | "client"
  onDecide={...}
/>
```

- Layout: two panes side by side on desktop/tablet widths — left pane the submitted image, right pane the generated OCR text — stacking vertically on narrow/mobile widths. This is the same component instance used for §2.4, parameterized by mode="client" so it renders Approve/Reject controls and shows the correctionist's prior decision as read-only context above the controls.

- Full-view controls: an “Expand” / full-screen icon on the image pane opens the image alone, at full resolution with pinch/scroll zoom, in a modal/lightbox. A separate “Expand” control on the OCR pane opens the OCR text alone, full-width, in a scrollable modal — useful for a client who wants to read or copy a long page of text without the image taking up half the screen. The two controls are independent: expanding one does not affect the other, and closing the expanded view returns to the split-screen side-by-side layout.
- Per-page navigator: thumbnail strip or page-number jump control (a client reviewing a 300-page book needs to move through pages quickly) with status color-coding (approved/pending/redo/rejected) so the client can spot-check rather than review every single page.
- Job dashboard header shows the JobReviewSummary counts live (websocket or polling), consistent with the roster/live-tracking update from §1.4.

5.5 Android (Kotlin) / iOS (Swift) changes

[DECISION NEEDED]: Confirm whether the client role is a native-app persona at all today, or web-only. The landing page's “Open client portal” links to the web app, suggesting clients may be web-first; if so, this review screen may only need to ship on web. If clients do use the native apps, mirror the same SubmissionReviewPane-equivalent screen described in §2.5/§2.6 for correctionists, with an Approve/Reject control set instead of Approve/Redo, including the same independent full-view behavior for image and OCR text.

5.6 Edge cases & QA checklist

- Client rejecting a submission that a correctionist already approved is clearly surfaced as an override, with the correctionist's original decision still visible for audit/trust purposes.
- Large jobs (hundreds of pages) — verify the per-page review UI is paginated/virtualized, not loading all images+OCR text at once.
- Client reject triggers whatever downstream dispute/refund/redo flow is decided in §5.3 — write an explicit state machine diagram before implementation.
- Review summary counts stay consistent under concurrent review activity (client and correctionists reviewing simultaneously).
- Verify that expanding the image pane to full-view and expanding the OCR pane to full-view both work correctly on the same submission in the same session, and that neither expanded state leaks into the next submission when the reviewer auto-advances.

6. NEW (Rev. 2) — OCR Preview Everywhere & Split-Screen Full-View Mode

This section consolidates a requirement that was previously scattered across §2.4 and §5.4 into one explicit, cross-cutting specification, because it applies to more surfaces than just the correctionist and client review screens.

6.1 What changes and why

Wherever a submitted image is shown anywhere in the product — the worker's own submission history, the correctionist review queue, and the client's per-page review screen — the OCR text generated from that image must be shown as a first-class preview alongside it, not buried behind a separate tap or hidden until a reviewer explicitly requests it. And wherever an image and its OCR are shown together, both must be independently expandable to a dedicated full-screen view.

Two concrete product requirements follow from this:

- 1. OCR preview on every submission card. Any list or grid of a worker's submitted images (their own history, an admin's audit view, a correctionist's queue before opening a submission) shows a short OCR text snippet under/beside the thumbnail once OCR has completed, plus a status chip (“Processing OCR...”, “OCR ready”, “OCR failed — will retry”). This lets anyone scanning a list get a sense of extraction quality without opening every submission individually.
- 2. Split-screen review with independent full-view. Any screen where a reviewer (correctionist, client, or admin) opens a single submission to act on it shows the image and its OCR text side by side (stacked on mobile), and each side has its own expand/full-view control, per §2.4 and §5.4 above.

6.2 Data model impact

No new fields are required beyond `Submission.ocrResult` (§2.2) and `Submission.qualityCheck` (§3.2), which already carry everything needed to render both the snippet and the full view. The only addition is a derived, UI-facing status the clients compute from existing fields rather than a new backend field:

```
// derived client-side, not stored – computed from Submission fields
type OcrPreviewState =
  | "processing"    // ocrResult is absent, upload succeeded
  | "ready"         // ocrResult present
  | "failed";       // backend flags OCR failure on this submission (see §6.3)
```

6.3 Backend changes

- Ensure `GET /jobs/:id/review-queue` (§2.3) and `GET /jobs/:id/submissions` (§5.3) both return the full `ocrResult` object (not a truncated/omitted version) so clients can render both the list-view snippet and the full-view text from the same payload, without a second round trip.
- Add a worker-facing endpoint, `GET /workers/me/submissions`, if one does not already exist, so the worker's own app/web view can list their submissions with OCR preview status — this is a read-only, own-data-only endpoint distinct from the correctionist/client review endpoints (no review actions are exposed here).
- If OCR fails outright (engine error, unreadable image that nonetheless passed the on-device quality filter), set a clear failure flag/reason on the `Submission` rather than leaving `ocrResult` permanently absent, so the UI can show “OCR failed — will retry” instead of an indefinite “processing” spinner.

6.4 Web (TypeScript) changes

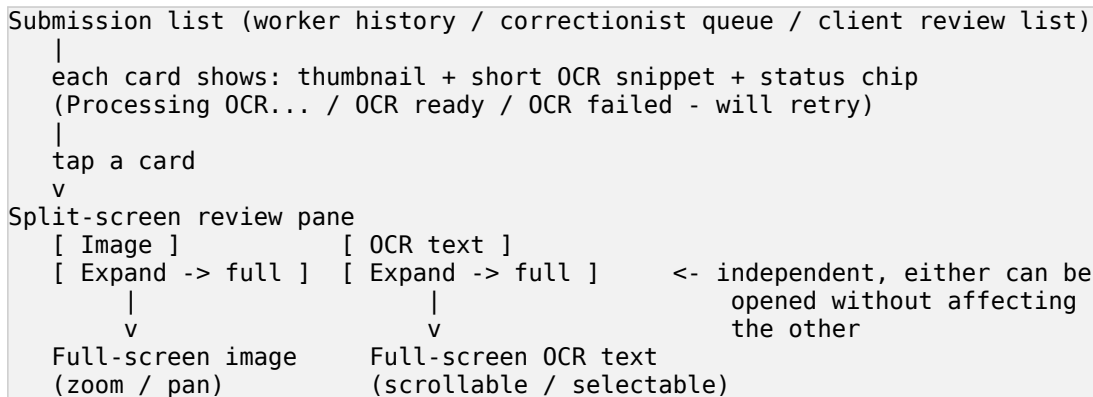
- Submission list/grid component (used in the worker's own history, admin audit views, and as the entry list before opening the correctionist/client review pane) renders a short OCR snippet (first ~2 lines of `ocrResult.text`, or a “Processing OCR...” chip) under each thumbnail.
- `SubmissionReviewPane` (§2.4/§5.4) is the single shared component for the split-screen, dual-full-view experience — implemented once, reused by `mode="correctionist"` and `mode="client"`, and reusable again for any future admin audit view without rebuilding the layout.

- Full-view modal for the image pane: renders the image alone at full resolution with zoom/pan; full-view modal for the OCR pane: renders the OCR text alone, selectable/copyable, full width. Both are simple overlay states on top of the same split-screen component — no route change, no data refetch.

6.5 Android (Kotlin) / iOS (Swift) changes

- Worker's own submissions/history screen: add the OCR snippet + processing/ready/failed status chip to each submission card, sourced from the new GET /workers/me/submissions endpoint (§6.3).
- Correctionist and (if applicable per §5.5) client review screens on native: the same independent full-view behavior as web — a tap/expand on the image opens a full-screen zoomable image viewer; a tap/expand on the OCR panel opens a full-screen, selectable text view. Implemented as two separate presented view controllers/screens layered over the split-pane review screen, matching the web modal pattern conceptually.

6.6 UX flow summary



6.7 Edge cases & QA checklist

- OCR snippet correctly truncates long text without breaking mid-word/mid-line in a jarring way; snippet updates from “Processing...” to real text without requiring a manual page refresh (poll or websocket-driven).
- OCR failed state is visually distinct from “still processing” — a reviewer should never mistake a permanently failed OCR job for one that just needs more time.
- Expanding the image full-view and then the OCR full-view in the same session (and switching back and forth) does not lose scroll position, zoom level being reset is acceptable but should not crash or blank the pane.
- On narrow/mobile widths, the stacked (non-split) layout still exposes both expand controls, not just one.
- Worker's own submission history never exposes reviewer-only actions (approve/redo/reject) — it is read-only, OCR-preview-only, consistent with the anonymity/least-privilege principle in §0.1.

7. Cross-Cutting Architecture Summary

7.1 Updated high-level flow (all changes combined) — UPDATED (Rev. 2)

1. Client posts a "mixed" job with `capacity.mode = "unlimited"` and `unitOfWork = { kind: "page", totalUnits: 300 }` (e.g., scan a 300-page book)
2. Any number of workers accept as "collectionist" (§1 + §2)
 - > each claims units, opens Capture flow (§3):
 - Capture -> Preview -> On-device quality filter (edge-to-edge / sharpness / exposure)
 - > FAIL (incl. not edge-to-edge): retake immediately, HARD BLOCK, never leaves device
 - > PASS: upload Submission + QualityCheckResult
3. Backend re-validates quality (§3.4), triggers OCR, Submission -> status "pending_review"
 - > as soon as OCR completes, the submission's OCR preview becomes visible in the worker's own history AND in the correctionist queue (§6)
4. Any eligible worker accepts the same job as "correctionist" (§1 + §2)
 - > Review queue: image + OCR side by side, each independently full-viewable (§2.4, §6)
 - > Approve -> status "approved" (payout releases to collectionist, §5.3)
 - > Redo -> unit re-opened, back into collectionist queue
5. Client opens their per-page review dashboard (§5, §6)
 - > sees image + OCR side by side, correctionist's decision as read-only context
 - > Approve (confirms) or Reject (overrides -> dispute/refund flow)
6. Admin console (§4, internal-only) monitors the whole pipeline: quality-filter telemetry, queue depths, redo rates, payouts, disputes

7.2 Shared component strategy

- packages/contracts is the single source of truth for Job, JobAssignment, Submission, ReviewEvent, QualityCheckResult shapes. Web consumes these TS types directly; Android/iOS keep equivalent Kotlin classes / Swift structs in lockstep.
- Web: one <SubmissionReviewPane> React component, parameterized by reviewer role, used for the correctionist screen (§2.4), the client screen (§5.4), and any future admin audit view — including the dual independent full-view behavior from §6.
- Android/iOS: one shared review-screen pattern (zoomable image + confidence-highlighted OCR text + role-appropriate action buttons + independent full-view on each pane), even if not literally shared code across Kotlin/Swift.
- Quality filter: same conceptual OpenCV pipeline (Canny/contours for edges, Laplacian variance for blur, histogram for exposure) on both native platforms; keep threshold constants in a shared, documented config (ideally remote-config-driven so thresholds can be tuned without an app store release).

7.3 Suggested rollout order

- 1. packages/contracts updates (§1.2, §2.2, §3.2, §5.2) — land first; everything else depends on the shared model.
- 2. Backend: JobAssignment refactor + unlimited-acceptance logic (§1.3) — foundational, low UI risk, can ship behind a feature flag.
- 3. Capture QA pipeline (§3), including the hard edge-to-edge reject gate — can be built and tested independently on both native apps in parallel with backend work.
- 4. Collectionist/Correctionist roles + review queue (§2), including OCR-preview cards (§6.4/§6.5) — depends on #1 and #2.
- 5. Client per-page review (§5) and the shared full-view split-screen behavior (§6) — depends on #4 (reuses its review-pane component).

- 6. Admin scope/access tightening + new dashboards (§4) — can happen in parallel; mostly access-control and internal-tooling work.

7.4 Open decisions requiring product sign-off

#	Decision	Recommendation given
1	Escrow model for unlimited jobs: per-unit price vs. pooled/pro-rata	Per-unit pricing (§1.3)
2	Correctionist eligibility bar (who's allowed to review)	Vetting/qualification step, admin-approved (§2.3)
3	Payout trigger: on correctionist approval vs. client approval	Correctionist approval releases payout; client reject triggers dispute flow after the fact (§5.3)
4	Admin portal public visibility (footer link)	Remove public link, keep internal-only access; native apps get no admin surface at all (§4.2)
5	Is “client” a native app persona or web-only today	Confirm with team; assume web-only unless stated otherwise (§5.5)

8. Master QA / Definition-of-Done Checklist

- packages/contracts updated and versioned; web + mobile apps pinned to the new version.
- Unlimited job acceptance verified under concurrency load.
- Collectionist and correctionist role flows fully navigable on web, Android, iOS.
- Correctionist cannot review own submissions.
- On-device quality filter integrated on Android (OpenCV/ML Kit) and iOS (VisionKit/OpenCV), with shared threshold config.
- Edge-to-edge boundary check confirmed as a hard reject (no upload path exists for a capture that fails it) across every capture entry point.
- Preview → retake loop tested on low-end devices for performance.
- Server-side quality re-check implemented as defense in depth.
- OCR pipeline wired from Submission upload through to correctionist queue, client review, and the worker's own submission history preview.
- OCR preview (snippet + status chip) visible on every submission card across worker history, correctionist queue, and client review list.
- Split-screen review pane live for both correctionist and client, each with independent “view full” controls on the image pane and the OCR pane.
- Client override (reject after correctionist approval) triggers the agreed dispute/refund flow.
- Admin console access tightened; public footer link decision finalized with product owner.
- Admin telemetry dashboards for quality-filter rejections and pipeline queue depth shipped.
- All open decisions in §7.4 formally signed off before final release.

End of specification. Questions or scope disputes should be raised against this document directly so the whole team stays aligned — please comment/PR against this file in docs/ rather than re-deriving requirements verbally.